code.presso

평가 결과
개인 리포트

이름

손덕인

이메일 dlwmznzl@gmail.com

소속 CJ4DPLEX

평가 정보	
평가명	MLOps 엔지니어_260405
평가 트랙 1	클라우드 & 분산 시스템 아키텍처 - 중급
평가 트랙 2	클라우드 컴퓨팅 - 중급
평가 트랙 3	Python - 전문가
평가 시작일	2026년 4월 4일
평가 소요 시간	2시간 19분 30초

Copyright © 2026 Codepresso, Inc. All Rights Reserved.

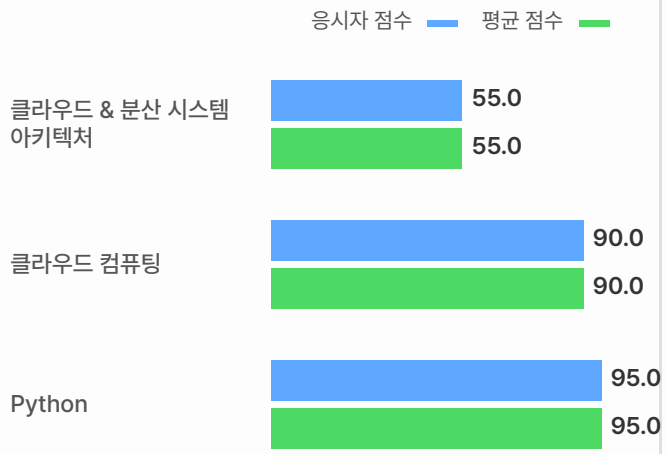
code.presso

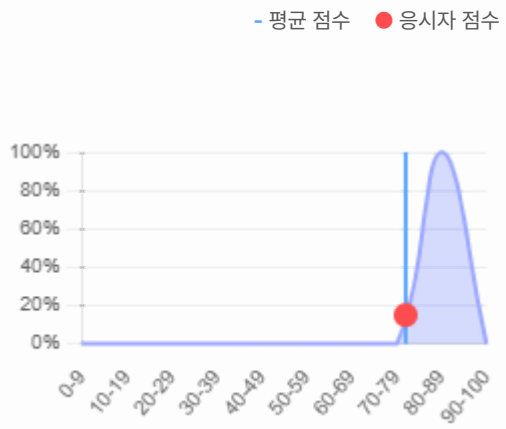
평가 결과 개인 리포트

역량 평가 개요

MLOps 엔지니어_260405 / 개요

80.0 / 100	80%	13 / 16	5 / 5
점수 / 총점	백분위	취득한 역량의 수	부정행위 지표





클라우드 & 분산 시스템 아키텍처 / 중급

중급

역량 보완 필요	55 / 100	2 / 5
역량 취득 상태	점수 / 총점	취득한 역량의 수

클라우드 컴퓨팅 / 중급

중급

역량 확보	90 / 100	4 / 4
역량 취득 상태	점수 / 총점	취득한 역량의 수

Copyright © 2026 Codepresso, Inc. All Rights Reserved.

code.presso

평가 결과 개인 리포트

역량 평가 개요

Python / 전문가

전문가	역량 확보	95 / 100	7 / 7
	역량 취득 상태	점수 / 총점	취득한 역량의 수

Copyright © 2026 Codepresso, Inc. All Rights Reserved.

평가 결과 개인 리포트

클라우드 & 분산 시스템 아키텍처 중급 역량 평가 상세 결과

역량	역량 확보 여부	역량 설명
분산 시스템 설계	△	본 역량은 확장 가능한 분산 환경을 구축하기 위한 설계 능력을 검증합니다. 분산 시스템의 기초 개념을 바탕으로 노드 간 상호작용 원리를 이해하고, 적절한 설계 패턴을 적용하여 아키텍처를 구조화하는 능력을 평가합니다. 또한, 트랜잭션 관리 심화 기술을 통해 복잡한 네트워크 상황에서도 데이터 정합성을 보장하는 실무 역량을 포함합니다.
고성능 시스템 아키텍처 설계	O	본 역량은 대규모 트래픽 환경에서도 안정적인 응답 속도를 유지하기 위한 최적화 능력을 검증합니다. 성능 측정 및 분석을 통해 시스템 병목 구간을 정확히 진단하고, 캐싱 전략 심화 기술을 적용하여 데이터 접근 속도를 극대화하는 능력을 평가합니다. 또한, 비동기 처리 심화 설계 역량을 바탕으로 시스템 간 결합도를 낮추고 처리 효율을 최적화하는 실무 능력을 포함합니다.
고가용성 시스템 설계 및 운영	△	본 역량은 서비스의 연속성을 보장하기 위해 중단 없는 시스템을 설계하고 운영하는 능력을 검증합니다. 고가용성 확보 전략을 수립하여 단일 장애 지점(SPOF)을 제거하고, 로드밸런싱을 통해 트래픽을 효율적으로 분산하여 시스템 부하를 관리하는 능력을 평가합니다. 또한, 장애 복구 전략을 바탕으로 비상 상황 발생 시 서비스를 신속하게 정상화하는 실무 대응 역량을 포함합니다.
클라우드 네이티브 아키텍처 설계	O	본 역량은 클라우드 환경에 최적화된 유연하고 탄력적인 시스템을 구축하는 능력을 검증합니다. 클라우드 네이티브 개념을 바탕으로 MSA 및 DevOps 원칙을 이해하고, 컨테이너 기술을 활용하여 애플리케이션의 배포와 관리 효율성을 극대화하는 능력을 평가합니다. 또한, 서버리스 컴퓨팅 설계를 통해 인프라 관리 부담을 최소화하고 자원을 효율적으로 운영하는 실무 역량을 포함합니다.
아키텍처 평가 및 개선	—	본 역량은 시스템의 현재 상태를 객관적으로 진단하고 지속 가능한 발전 방향을 설계하는 능력을 검증합니다. 아키텍처 평가 방법과 품질 속성을 활용하여 시스템의 구조적 적합성을 판별하고, 아키텍처 개선 전략을 통해 도출된 문제점을 체계적으로 해결하는 능력을 평가합니다. 또한, 누적된 기술 부채를 관리하고 중장기적 기술 로드맵을 수립하여 비즈니스 변화에 유연하게 대응하는 실무 역량을 포함합니다.

O 역량을 확보하고 있음
△ 개념에 대한 이해는 있으나, 산업에서의 적용 및 활용 역량은 보완이 필요함
— 역량을 확보하고 있지 않음

Copyright © 2026 Codepresso, Inc. All Rights Reserved.

평가 결과 개인 리포트

나의 스킬 역량 확보 결과

총평

Cloud & Distributed Systems Architecture 트랙에서 응시자는 고성능 시스템 아키텍처 설계와 클라우드 네이티브 아키텍처 설계에서 캐시 전략, 비동기 처리, 불변 인프라와 컨테이너 보안 개념을 이해하고 있습니다. 이를 통해 처리량과 응답 시간을 고려한 구조를 구상하는 역량이 확인됩니다.

다만 분산 시스템 설계에서는 고성능 아키텍처 설계 패턴과 분산 트랜잭션 보상 전략 설계 역량이 부족하며, 고가용성 시스템 설계 및 운영에서도 대규모 트래픽 로드밸런싱과 장애 복구 전략 설계 능력이 충분히 드러나지 않습니다. 아키텍처 평가 및 개선에서는 이벤트 기반 아키텍처의 확장성과 일관성 평가, Canary·병렬 운영 전략 이해가 미흡해 구조 진단과 개선 역량 보완이 필요합니다.

전체적으로 이 트랙에서는 일부 성능·클라우드 네이티브 영역에서 강점이 있으나, 분산 시스템 전반의 구조 설계와 고가용성, 아키텍처 평가 역량이 부족한 상태입니다. 분산 패턴과 보상 트랜잭션, 로드밸런싱과 DR 전략, 품질 속성 기반 평가 기법을 중심으로 학습을 강화하는 것이 바람직합니다.

강점

- **고성능 시스템 아키텍처 설계**: 고성능 시스템 아키텍처 설계 역량에서 캐시 성능 문제를 진단하고 분석하는 개념을 이해하고 있으며, 고빈도 거래 환경을 가정한 캐싱 전략 설계와 비동기 처리 패턴을 통해 처리량과 응답 시간을 안정적으로 유지하는 구조를 떠올릴 수 있는 역량이 확인됩니다. 실시간 부하를 캐시와 비동기 흐름으로 분산하는 설계 관점이 드러납니다.

- **클라우드 네이티브 아키텍처 설계**: 클라우드 네이티브 아키텍처 설계 역량에서 불변 인프라 특징과 컨테이너 보안 강화 방법을 이해하고 있어, 이미지 기반 배포와 자동화를 전제로 한 구조를 떠올릴 수 있는 역량이 확인됩니다. 다만 서버리스 기반 DevOps 파이프라인 설계 전략 이해는 제한적이어서, 서버리스 환경에서 배포와 운영을 통합하는 설계 경험을 넓힐 여지가 있습니다.

약점

- **분산 시스템 설계**: 분산 시스템 설계 역량에서 분산 환경의 시간 동기화 문제 개념은 이해하고 있으나, 고성능 시스템의 아키텍처 설계 패턴을 선택하고 적용하는 역량은 충분히 드러나지 않습니다. 분산 트랜잭션 보상 전략 설계에서도 일관된 전략을 구성하는 능력이 부족해, 확장성과 일관성을 함께 고려한 구조 설계 역량을 보완할 필요가 있습니다.

Copyright © 2026 Codepresso, Inc. All Rights Reserved.

평가 결과 개인 리포트

나의 스킬 역량 확보 결과

- **고가용성 시스템 설계 및 운영**: 고가용성 시스템 설계 및 운영 역량에서 모니터링과 경보 전략 개념은 이해하고 있으나, 대규모 트래픽 환경의 로드밸런싱 전략을 체계적으로 선택하고 장애 복구 전략을 설계하는 역량은 부족합니다. 트래픽 분산과 복구 시나리오를 구조적으로 연결해 단일 장애 지점을 제거하는 설계 능력을 강화할 필요가 있습니다.

- **아키텍처 평가 및 개선**: 아키텍처 평가 및 개선 역량에서 이벤트 기반 아키텍처의 확장성과 일관성을 평가하는 개념 이해가 부족하며, Canary·병렬 운영과 같은 점진적 배포 전환 전략을 활용해 위험을 통제하는 접근도 충분히 드러나지 않습니다. 품질 속성을 기준으로 구조를 진단하고 개선 경로를 설계하는 역량을 전반적으로 강화할 필요가 있습니다.

Copyright © 2026 Codepresso, Inc. All Rights Reserved.

평가 결과 개인 리포트

클라우드 컴퓨팅 중급 역량 평가 상세 결과

역량	역량 확보 여부	역량 설명
클라우드 컴퓨팅 개념과 구조	O	본 역량은 클라우드 컴퓨팅의 기본 개념과 구조적 원리를 이해하는 능력을 검증합니다. 클라우드 환경을 구성하는 핵심 요소와 그 상호 연관성을 파악하고, 클라우드 관리 및 보안의 기본 요소를 인식하는 것이 포함됩니다.
확장성과 복원력을 갖춘 클라우드 아키텍처 설계	O	본 역량은 확장성과 복원력을 갖춘 클라우드 아키텍처를 설계하는 능력을 검증합니다. 자동 확장, 로드 밸런싱, 멀티 AZ 설계 등의 개념을 적용하여 장애 복구와 재해 대응이 가능한 시스템을 구축하는 것이 포함됩니다.
실무 중심 클라우드 보안 및 규제 준수	O	본 역량은 실무 중심의 클라우드 보안 및 규제 준수 역량을 검증합니다. IAM 역할 및 정책 설정, 보안 네트워크 아키텍처 구축, 데이터 암호화 및 접근 제어 적용, 안전한 클라우드 운영을 위한 규제 요구사항 이해가 포함됩니다.
클라우드 네트워크 및 성능 최적화	O	본 역량은 클라우드 네트워크를 효율적이고 안정적으로 설계·관리·최적화하는 능력을 검증합니다. 보안 네트워크 구조 설계, DNS와 CDN을 활용한 트래픽 분산, 네트워크 성능 모니터링 및 문제 해결, 비용 효율적 네트워크 전략 수립이 포함됩니다.

O 역량을 확보하고 있음
△ 개념에 대한 이해는 있으나, 산업에서의 적용 및 활용 역량은 보완이 필요함
— 역량을 확보하고 있지 않음

Copyright © 2026 Codepresso, Inc. All Rights Reserved.

평가 결과 개인 리포트

나의 스킬 역량 확보 결과

총평

Cloud Computing 트랙에서 응시자는 클라우드 컴퓨팅 개념과 구조 역량을 통해 멀티리전·Failover·DR 개념과 자동 사용량 측정·과금 모델을 이해하고 있으며, KMS·SSE·공개 접근 점검을 연계해 데이터 보호를 설계하는 능력을 보여주었습니다. 기본 구조와 운영 메커니즘을 함께 고려하는 역량이 확인됩니다.

확장성과 복원력을 갖춘 클라우드 아키텍처 설계와 클라우드 네트워크 및 성능 최적화 역량에서는 다중 데이터센터와 멀티리전 구성을 전제로 자동 확장, 이중 로드밸런싱, 자동 Failover를 결합해 비즈니스 연속성을 확보하는 설계가 드러납니다. 실무 중심 클라우드 보안 및 규제 준수 역량에서도 암호화, RBAC, 로그와 규제 기준을 연계해 보안과 거버넌스를 함께 설계하는 능력이 확인됩니다.

전체적으로 이 트랙에서는 성능, 복원력, 보안, 비용 구조를 함께 고려한 클라우드 아키텍처 설계 역량이 강점으로 나타납니다. 다중 리전과 멀티클라우드 환경에서 네트워크와 보안, DR을 통합적으로 설계할 수 있는 기반이 갖추어져 있어, 대규모 클라우드 환경에서도 구조 설계와 운영에 기여할 수 있는 수준으로 평가됩니다.

강점

- **클라우드 컴퓨팅 개념과 구조**: 클라우드 컴퓨팅 개념과 구조 역량에서 단일 리전 장애와 멀티리전·Failover·DR 개념을 이해하고 있으며, 자동 사용량 측정과 과금 모델을 파악해 아키텍처와 비용 구조를 함께 고려하는 능력이 확인됩니다. 비암호화 민감 데이터에 대해 KMS·SSE·공개 접근 점검을 연계해 데이터 보호를 설계하는 역량도 보유하고 있습니다.

- **확장성과 복원력을 갖춘 클라우드 아키텍처 설계**: 확장성과 복원력을 갖춘 클라우드 아키텍처 설계 역량에서 다중 데이터센터와 멀티클라우드 환경을 전제로 한 이중 로드밸런싱과 자동 복구 구조를 설계하는 능력이 확인됩니다. 수요 기반 자동 확장과 글로벌 트래픽 분산을 통해 장애 복원과 용량 관리를 정책 기반으로 운영하려는 관점이 드러나며, 이를 비즈니스 연속성과 직접 연결하는 역량을 갖추었습니다.

- **실무 중심 클라우드 보안 및 규제 준수**: 실무 중심 클라우드 보안 및 규제 준수 역량에서 전송 구간과 저장 구간 암호화를 플랫폼 키 관리 서비스와 연계해 설계하고, RBAC와 최소 권한 원칙으로 관리자 권한을 통제하는 능력이 확인됩니다. 중앙 집중 로그와 정기적인 규제 기준 점검을 통해 거버넌스를 유지하려는 관점도 드러나, 보안과 컴플라이언스를 구조적으로 바라보는 역량을 보유하고 있습니다.

Copyright © 2026 Codepresso, Inc. All Rights Reserved.

평가 결과 개인 리포트

나의 스킬 역량 확보 결과

- **클라우드 네트워크 및 성능 최적화**: 클라우드 네트워크 및 성능 최적화 역량에서 글로벌 CDN 아키텍처와 지연 시간 기반 라우팅, 엣지 캐싱을 활용해 전 세계 사용자 응답 시간을 개선하는 능력이 확인됩니다. 비용 효율 DR과 멀티리전 구성·자동 Failover를 결합해 가용성과 비용을 함께 고려하고, 사설 연결과 암호화 통신, 지표 기반 튜닝을 통해 보안과 성능을 동시에 관리하는 역량을 갖추었습니다.

약점
확인된 약점이 없습니다.

Copyright © 2026 Codepresso, Inc. All Rights Reserved.

평가 결과 개인 리포트

Python 전문가 역량 평가 상세 결과

역량	역량 확보 여부	역량 설명
데코레이터	O	본 역량은 Python에서 데코레이터를 이해하고 구현하는 능력을 검증합니다. 데코레이터 정의, 인자를 포함한 데코레이터 적용, 데코레이터 중첩, 클래스 기반 데코레이터와 같은 대표적인 내용을 포함합니다.
고급 Python 모듈	O	본 역량은 고급 Python 모듈을 이해하고 활용하는 능력을 검증합니다. 알고리즘, 자료구조, 수학 연산, 날짜 및 시간 처리와 관련된 모듈과 같은 내용을 포함합니다.
람다	O	본 역량은 Python에서 람다 함수를 정의하고 활용하는 능력을 검증합니다. 매개변수 처리, 반환값 처리, 간결한 연산을 위한 람다 함수 활용과 같은 내용을 포함합니다.
파일 입출력	O	본 역량은 Python에서 파일 입출력 작업을 수행하는 능력을 검증합니다. 파일 읽기, 파일 쓰기, 파일 모드 관리, 파일 처리 중 예외 처리와 같은 내용을 포함합니다.
JSON	O	본 역량은 Python에서 JSON 데이터를 파싱하고 처리하는 능력을 검증합니다. JSON 데이터 읽기 및 쓰기, JSON 구조 이해, JSON 데이터 타입 처리를 포함합니다.
멀티프로세싱	O	본 역량은 Python에서 다중 프로세스를 생성하고 관리하는 능력을 검증합니다. 프로세스 생성 및 관리, 프로세스 간 통신, 동기화 기법, 멀티프로세싱 환경에서의 예외 처리를 포함합니다.
멀티스레딩	O	본 역량은 Python에서 멀티스레딩을 구현하는 능력을 검증합니다. 스레드 생성 및 관리, 스레드 간 통신, 동기화 기법, 스레드 프로그램에서의 예외 처리를 포함합니다.

O 역량을 확보하고 있음
△ 개념에 대한 이해는 있으나, 산업에서의 적용 및 활용 역량은 보완이 필요함
— 역량을 확보하고 있지 않음

Copyright © 2026 Codepresso, Inc. All Rights Reserved.

평가 결과 개인 리포트

나의 스킬 역량 확보 결과

총평

본 응시자는 Python 트랙에서 고급 Python 모듈과 파일 입출력, 데코레이터와 람다, 멀티프로세싱과 멀티스레딩 전반에 걸쳐 요구 역량을 충족했습니다. 표준 라이브러리를 목적에 맞게 선택하고, 자원 관리를 컨텍스트 기반으로 처리하며, 동시성 구조를 설계하는 능력이 확인됩니다.

또한 JSON 처리에서도 파일 포인터 제어와 안전한 덮어쓰기를 통해 데이터 무결성을 유지하고, 데코레이터를 활용해 접근 제어와 로깅을 구조적으로 분리하는 등 언어 기능을 설계 관점에서 활용하는 역량이 드러납니다. 멀티프로세싱과 멀티스레딩에서는 작업 분할과 생명주기 관리를 안정적으로 수행하고 있습니다.

전체적으로 Python 트랙에서는 고급 모듈 활용, 파일 기반 데이터 처리, 함수형 기능, 동시성 제어를 아우르는 구현 역량이 확인됩니다. 요구된 기능을 모듈화하고 여러 실행 단위를 조합해 동작시키는 능력이 검증되어, 실무 환경에서도 Python을 활용해 안정적인 서비스를 구성할 수 있는 기반을 갖추었다고 판단됩니다.

강점

- **데코레이터**: 데코레이터 역량에서 함수를 인자로 받아 감싸는 구조를 이해하고, 접근 제어와 로깅처럼 여러 함수에 공통 적용되는 동작을 데코레이터로 분리하는 능력이 확인됩니다. 래퍼에서 인자를 전달하고 반환값을 유지하며, 예외 상황을 함께 처리하는 등 교차 관심사를 구조적으로 관리하는 역량을 갖추었습니다.

- **고급 Python 모듈**: 고급 Python 모듈 역량에서 표준 라이브러리를 상황에 맞게 선택해 활용하는 능력이 확인됩니다. 알고리즘과 수치 계산, 통계 처리, 멀티프로세싱과 멀티스레딩을 연계해 문제를 구조적으로 분해하고, 컨텍스트 관리로 자원 수명을 명확히 구분하는 설계가 드러납니다. 요구된 기능을 여러 모듈을 조합해 안정적으로 구현할 수 있는 역량을 갖추었습니다.

- **람다**: 람다 역량에서 단일 목적의 간결한 연산을 람다 표현식으로 정의하고, 필요한 인자를 받아 불리언 판단에 활용하는 능력이 확인됩니다. 접근 제어와 같이 조건 판단이 중요한 위치에 람다를 배치해 로직을 분리하고, 주변 구조와 조화롭게 통합하는 등 람다를 적절한 범위에서 활용하는 역량을 보유하고 있습니다.

- **파일 입출력**: 파일 입출력 역량에서 컨텍스트 관리를 통해 파일을 안전하게 열고 닫으며, 읽기·쓰기 모드를 목적에 맞게 선택하는 능력이 확인됩니다. 대용량 데이터를 줄 단위로 처리해 메모리 사용을 통제하고, 오류 정보를 별도 로그로 남기는 등 파일을 데이터 처리 흐름 속에 자연스럽게 통합하는 역량을 갖추었습니다.

- **JSON**: JSON 역량에서 파일을 열고 JSON 데이터를 파이썬 구조로 변환한 뒤 수정하고 다시 저장하는 흐름을 안정적으로 구성하는 능력이 확인됩니다. 컨텍스트 관리로 파일 수명을 관리하고, 포인터 이동과 잘림 처리를 통해 이전 내용이 남지 않도록 제어하는 등 실제 환경에서 요구되는 JSON 파일 입출력 역량을 보유하고 있습니다.

Copyright © 2026 Codepresso, Inc. All Rights Reserved.

평가 결과 개인 리포트

나의 스킬 역량 확보 결과

- **멀티프로세싱**: 멀티프로세싱 역량에서 작업을 독립 단위로 나누고 프로세스 풀을 통해 병렬 실행하는 구조를 설계하는 능력이 확인됩니다. 프로세스 생명주기를 컨텍스트 기반으로 관리하며, 풀과 매핑 기능을 활용해 여러 입력을 균등하게 분배하는 등 프로세스 생성과 정리를 안정적으로 수행할 수 있는 역량을 보유하고 있습니다.

- **멀티스레딩**: 멀티스레딩 역량에서 작업을 여러 스레드로 나누고 생성·시작·대기를 명확히 구분해 제어하는 능력이 확인됩니다. 공유 자료와 파일 접근 구간에 잠금을 적용해 경쟁 상태를 예방하고, 정해진 위치에 결과를 모으는 구조를 설계하는 등 스레드 기반 병렬 처리를 안전하게 구성하는 역량을 보유하고 있습니다.

약점

확인된 약점이 없습니다.

Copyright © 2026 Codepresso, Inc. All Rights Reserved.

제출 코드 정적분석 결과

문제 1 _ main.py

개요

이 코드는 데코레이터로 함수 실행 결과를 파일에 기록하는 구조로, 중복이 적고 흐름이 단순해 유지보수가 쉽습니다. 다만 실행 중 오류가 나면 오류를 기록한 뒤에도 실패 사실을 호출자에게 명확히 전달하지 않아, 테스트나 운영에서 문제를 놓칠 위험이 있습니다. 오류 발생 시 동작을 더 명확히 하면 신뢰성과 품질 관리에 도움이 됩니다.

지표명	결과	설명
DUPLICATIONS	PASS	로깅 기능이 데코레이터로 분리되어 있어 동일한 로깅 로직을 여러 곳에 반복 작성하지 않습니다.
CYCLOMATIC_COMPLEXITY	PASS	함수 흐름이 단순하며, 조건 분기와 예외 처리 외에 복잡한 분기 구조가 거의 없습니다.
BUGS	WARNING	예외가 발생했을 때 로그만 남기고 예외를 다시 발생시키지 않아, 호출 측에서 실패를 정상 동작으로 오해할 수 있고 결과가 반환되지 않을 수 있습니다.
CODING_STYLE	PASS	구조가 간단하고 역할이 분리되어 전반적으로 읽기 쉬우며, 함수와 변수 이름이 동작을 잘 설명합니다.

Copyright © 2026 Codepresso, Inc. All Rights Reserved.

제출 코드 정적분석 결과

문제 2 _ main.py

개요

코드는 JSON 파일을 읽고 수정한 뒤 다시 저장하는 흐름이 함수로 나뉘어 있어 구조는 비교적 명확합니다. 다만 예외가 발생했을 때 파일이 정상적으로 닫히지 않을 수 있는 경로가 있고, 여러 프로세스가 동시에 같은 파일을 수정할 수 있어 결과가 불안정해질 위험이 있습니다. 또한 일부 들여쓰기/공백 스타일이 섞여 있어 유지보수 관점에서 가독성이 떨어질 수 있습니다.

지표명	결과	설명
DUPLICATIONS	PASS	파일 읽기/쓰기와 데이터 갱신 로직이 함수로 분리되어 있으며, 동일한 로직을 반복해서 작성한 부분이 두드러지지 않습니다.
CYCLOMATIC_COMPLEXITY	PASS	각 함수의 분기와 흐름이 단순한 편이며, 전체적으로 이해하기 쉬운 구조입니다.
BUGS	WARNING	json_file_manager에서 파일을 연 뒤 json.load 과정에서 예외가 발생하면 파일이 닫히지 않은 채로 종료될 수 있습니다. 또한 여러 프로세스가 같은 파일을 동시에 열어 수정할 수 있어, 실행 타이밍에 따라 파일 내용이 꼬이거나 일부 변경이 유실될 가능성이 있습니다.
CODING_STYLE	WARNING	들여쓰기 깊이와 공백 사용이 일부 구간에서 일관되지 않아 읽는 사람이 흐름을 따라가기 어려울 수 있습니다.

Copyright © 2026 Codepresso, Inc. All Rights Reserved.

제출 코드 정적분석 결과

문제 3 _ main.py

개요

코드는 파일별 처리를 함수로 분리하고, 데이터 형식 오류와 변환 오류를 로그로 남기는 등 전체 구조가 단순하고 이해하기 쉽습니다. 다만 예외를 너무 넓게 잡아 문제 원인을 파악하기 어려울 수 있고, 여러 프로세스가 동시에 같은 로그 파일에 쓰는 방식은 운영 중 기록 신뢰도를 떨어뜨릴 수 있습니다. 또한 사용되지 않는 요소와 스타일의 작은 불일치가 있어, 유지보수 관점에서 정리하면 더 깔끔해질 여지가 있습니다.

지표명	결과	설명
DUPLICATIONS	PASS	파일 처리 로직이 함수로 분리되어 있고, 동일한 코드 블록을 반복해서 복사한 흔적이 두드러지지 않습니다.
CYCLOMATIC_COMPLEXITY	PASS	주요 흐름이 입력 검증 → 변환 시도 → 계산 → 출력의 단순한 단계로 구성되어 있으며, 조건 분기와 반복이 과도하지 않습니다.
BUGS	WARNING	숫자 변환에서 모든 예외를 포괄적으로 처리하고 있어(예: except:) 실제 원인과 무관한 오류까지 같은 방식으로 숨겨질 수 있습니다. 또한 여러 프로세스가 동시에 같은 error.log 파일에 기록하므로 로그 내용이 섞이거나 일부가 누락되는 등 기록 품질이 떨어질 가능성이 있습니다.
CODING_STYLE	WARNING	사용되지 않는 import(defaultdict)가 포함되어 있어 코드가 불필요하게 복잡해 보일 수 있습니다. 또한 들여쓰기와 공백 사용이 일부 구간에서 일관되지 않아 읽는 사람이 구조를 빠르게 파악하기 어려울 수 있습니다.

Copyright © 2026 Codepresso, Inc. All Rights Reserved.

제출 코드 정적분석 결과

문제 4 _ main.py

개요

코드는 데코레이터로 사용자 접근을 통제하고 실패 횟수를 기록하려는 목적은 분명하지만, 실행에 직접 영향을 주는 문제가 포함되어 있습니다. 특히 스레드 안전을 위한 'lock'이 정의되지 않아 프로그램이 오류로 중단될 수 있고, 동일 이름의 함수가 중복 정의되어 동작을 예측하기 어렵습니다. 또한 조건 분기가 여러 갈래로 나뉘어 있어 규칙 변경 시 품질 저하나 실수로 이어질 가능성이 있습니다. 이러한 부분을 정리하면 안정성과 유지보수성이 개선됩니다.

지표명	결과	설명
DUPLICATIONS	FAIL	'access_control' 데코레이터 내부에서 'wrapper' 함수가 동일한 이름으로 중복 정의되어 있어, 어떤 함수가 실제로 실행되는지 이해하기 어렵고 유지보수가 어려워집니다.
CYCLOMATIC_COMPLEXITY	WARNING	접근 허용/차단/차단 임계치 처리 등이 여러 조건문으로 나뉘어 있어, 흐름을 한눈에 파악하기 어렵고 향후 규칙이 추가될 때 실수 가능성이 커질 수 있습니다.
BUGS	FAIL	스레드 동기화를 위해 사용한 'lock' 변수가 코드 내에서 정의되지 않아 실행 시 오류가 발생합니다. 또한 로그 메시지에 'ACCESS DENIE' 오타가 있어 운영 중 모니터링/감사 로그의 신뢰도를 떨어뜨릴 수 있습니다.
CODING_STYLE	WARNING	중첩된 함수 정의와 람다 사용 등으로 코드 구조가 불필요하게 복잡해 보이며, 들여쓰기/구조가 일관되지 않아 읽기와 점검이 어려울 수 있습니다.

Copyright © 2026 Codepresso, Inc. All Rights Reserved.

제출 코드 정적분석 결과

문제 5 _ main.py

개요

코드는 기능별로 함수가 나뉘어 있어 구조는 비교적 명확하지만, 여러 오타와 이름 불일치로 인해 실제 실행 시 오류가 발생할 가능성이 높습니다. 또한 조합 처리 과정에서 반복문이 중첩되고 한 함수에 여러 책임이 섞여 있어 유지보수 부담이 커질 수 있습니다. 전반적으로 가장 우선순위는 실행을 막는 오타/불일치와 결과 계산 흐름의 오류를 정리하는 것이며, 그 다음으로 예외 처리와 코드 정리(불필요한 주석 블록 정돈)를 통해 가독성과 안정성을 높이는 것이 좋습니다.

지표명	결과	설명
DUPLICATIONS	PASS	함수들이 조합 생성, 통계 계산, 파일 저장, 스레드 실행 등 역할별로 분리되어 있으며, 동일한 로직이 여러 곳에 반복되어 나타나는 부분은 두드러지지 않습니다.
CYCLOMATIC_COMPLEXITY	WARNING	process_combinations 함수에 반복문이 중첩되어 있고 (조합 반복 + 각 조합 원소 반복), 파일 저장까지 한 흐름에 묶여 있어 읽고 수정하기가 다소 어렵습니다.
BUGS	FAIL	실행 오류를 유발할 수 있는 오타와 이름 불일치가 존재합니다. 예: calculate_statistics에서 item["prodcut"] 키 사용, combinattions_data 변수명 오타, save_combinations_to_json에서 oepn 오타, process_combinations에서 calcurate_statistics/stas·stats 변수명 불일치, 결과 append가 내부 루프에 있어 product가 완성되기 전에 여러 번 기록될 수 있습니다.
CODING_STYLE	WARNING	이름 규칙과 예외 처리 방식이 일관되지 않습니다. 예를 들어 변수명(stas/stats)과 함수명(calcurate_statistics)이 혼재되어 있고, 파일 읽기에서 모든 예외를 한 번에 처리하는 형태(except:)라 문제 원인 파악이 어려울 수 있습니다. 또한 주석 처리된 코드 블록이 길게 남아 있어 핵심 로직 파악을 방해할 수 있습니다.

Copyright © 2026 Codepresso, Inc. All Rights Reserved.

제출 코드 정적분석 결과

문제 6 _ main.py

개요

코드는 리스트를 여러 조각으로 나눈 뒤 각 조각을 동시에 정렬하고, 마지막에 합치는 구조로 중복이 적고 전체 흐름도 단순합니다. 다만 결과를 반환하지 않고 출력에 의존해 활용 범위가 좁고, 빈 입력처럼 예외적인 상황에서 오류가 날 수 있어 안정성 측면의 보완이 필요합니다. 또한 이름 규칙이 일부 일관되지 않아 장기적으로 읽기 쉬운 코드 품질을 위해 정리하는 것이 좋습니다.

지표명	결과	설명
DUPLICATIONS	PASS	정렬, 분할, 병합 기능이 함수로 분리되어 있으며 동일한 로직을 반복해서 작성한 부분이 거의 보이지 않습니다.
CYCLOMATIC_COMPLEXITY	PASS	각 함수의 분기(조건 처리)가 제한적이며 전체 흐름이 단순해 이해와 변경이 비교적 쉽습니다.
BUGS	WARNING	정렬 결과를 함수에서 반환하지 않고 print로만 출력하여, 다른 코드에서 재사용하거나 테스트하기 어렵습니다. 또한 입력이 빈 리스트인 경우 분할 크기 계산에서 0으로 나누는 상황이 발생할 수 있습니다.
CODING_STYLE	WARNING	전역 상수처럼 보이는 'Num_Threads'가 일반적인 상수 표기(예: 대문자 스네이크 케이스)와 다르게 혼용되어 있어, 팀 단위 유지보수 시 규칙을 이해하는 데 시간이 더 걸릴 수 있습니다.

Copyright © 2026 Codepresso, Inc. All Rights Reserved.

평가 결과 개인 리포트

무결성 등급 설명

점수	판정 명칭	의미 및 상세 설명
5	부정행위 없음	평가 도중 의심스러운 행동이나 부정행위 기록 없이, 모든 규정을 철저히 준수했습니다.
4	경미한 의심	약간의 의심스러운 행동이 감지되었으나, 명백한 부정행위로 단정할 수 없는 수준입니다.
3	의심스러운 행동	부정행위로 의심될 만한 복수의 행동이 확인되었으며, 추가적인 수동 검증이 반드시 필요합니다.
2	명백한 시도	부정행위를 시도한 명확한 증거가 확인되었습니다.
1	부정행위 확인	명백한 부정행위가 발생하여 테스트 결과의 신뢰성이 상실되었습니다.

평가자의 관찰된 행동

관찰된 특이사항이 없습니다.

Copyright © 2026 Codepresso, Inc. All Rights Reserved.